RAPID: Re-Configurable Adder Tree Based Performance-Incentivized AI Digital CIM Macro

Akash Sankhe, Mukul Lokhande, *Member, IEEE*, Narendra Singh Dhakad, Radheshyam Sharma, and Santosh Kumar Vishvakarma, *Senior Member, IEEE*

***Abstract*—Rapidly rising AI applications have driven the need for energy-efficient, high-throughput SRAM-embedded Compute-in-Memory (CIM) macros. This article proposes RAPID-CIM, a reconfigurable digital CIM featuring an M8T bitcell and an area-efficient hierarchical adder tree, which addresses performance limitations in state-of-the-art (SoTA) designs. The proposed architecture achieves a maximum 16.7× improvement in energy efficiency and an 8.6× increase in compute density compared to SoTA designs at CMOS 65nm. RAPID-CIM achieves superior performance for diverse AI workloads with scalable bit-precision and sparsity-aware operations, reducing up to 3.2× operation cycles and a 30% memory bank usage and application accuracy within 97.85% Quality of Results (QoR). Thus, the proposed solution is a well-versed optimisation for resource-constrained Edge-AI platforms.**

***Index Terms*—IMC, edge-AI accelerators, reconfigurable precision, multiply-and-accumulate, and deep neural networks.**

I. Introduction

ARTIFICIAL Intelligence (AI) has become a significant part of human life with data-driven decision-making. The computational cost associated with AI algorithms limits the usage. The memory bottleneck has been the primary issue, especially given the heavy data dependency required for quick event-triggered responses. This motivates improving energy efficiency and compute density across applications. Prior work has focused on digital reconfigurable multi-precision AI architectures [1], [2], [3]; however, these architectures must be integrated with digital CIM (DCIM) accelerators to overcome the von Neumann bottleneck [4], [5]. Our approach focuses on resource-efficient bit-reconfigurable precision multiplication and accumulation (MAC) operations to enhance throughput in neural network (NN) computation. In contrast, prior works offer limited multi-precision scalability; for instance, Flex-PE [1] supports 4/8/16/32-bit operands, RD-CIM [3] targets 4/8/12/16-bit computations, while Flex-DCIM [2] enables bit-precision reconfigurability, albeit at the cost of latency and area-overhead.

Received 13 September 2025; revised 16 January 2026; accepted 29 March 2026. Date of publication 1 April 2026; date of current version 13 April 2026. This work was supported in part by the Special Manpower Development Program for Chip to Start-Up under Grant SMDP-C2S and in part by the Ministry of Electronics and Information Technology (MeitY), Govt. of India, under Grant EE-9/2/21 - R&DE. The review of this article was arranged by Associate Editor I. Vourkas. *(Corresponding author: Santosh Kumar Vishvakarma.)*

The authors are with the NSDCS Research Group, Department of Electrical Engineering, Indian Institute of Technology (IIT) Indore, Simrol 453552, India (e-mail: skvishvakarma@iiti.ac.in).

Digital Object Identifier 10.1109/TNANO.2026.3680063

SRAM-based DCIM is preferred for low access latency and no periodic refresh requirement. In contrast, analog CIM (ACIM) suffers from non-linearities and process variations. Many prior designs also exhibit low compute density due to bulky bitcells and transistor-heavy adder trees, which limit scalability and energy efficiency. Thus, Several digital compute-in-memory (DCIM) approaches replace transistor-heavy CMOS logic with more area-efficient pass-transistor logic (PTL). However, PTL inherently suffers from threshold voltage loss when transmitting logic levels. Specifically, an NMOS pass transistor cannot fully transmit a logic '1', i.e $V_{OH} = V_{DD}$ - $V_{th,n}$, while a PMOS pass transistor cannot fully transmit a logic '0', i.e $V_{OL} = |V_{th,p}|$). In contrast, NMOS devices pass a strong logic '0' ($V_{OL} = 0$) and PMOS devices pass a strong logic '1' ($V_{OH} = V_{DD}$), which motivates the use of complementary or transmission-gate-based PTL structures to mitigate threshold loss. Our approach uses a PMOS device in AND logic, where the critical output is logical '1'. DCIM also avoids data-conversion errors and poor voltage scaling to address this issue, but requires a decoupled read port to prevent bit flips caused by the simultaneous activation of word lines in shared SRAM columns. Our solution resolves this with wordline-independent CIM operation. The article presents a performance-enhanced reconfigurable DCIM macro, with key contributions as follows:

- *Novel M8T bitcell for DCIM array:* The proposed approach enables word-line-independent multiplication using a novel M8T bit cell, addressing the challenges of DCIM operations. The bit cell can function independently as both a storage element and a compute block, without affecting either functionality.
- *Novel FA14 T and HA11 T for Area-efficient RCA:* The novel 11 T half adder (HA11 T) and 14 T full adder (FA14 T) provide a full rail-to-rail swing and an area-efficient adder tree with a ripple carry adder (RCA). The reduction in the number of transistors contributes to a reduced area and enhanced compute density.
- *Enhanced Performance Reconfigurable DCIM Macro:* The proposed approach explores MUX-based reconfigurable adder tree architecture to address memory bank under-utilisation. The work shows significant improvements in 16.7× energy efficiency and 8.6× compute density. The proposed macro has been evaluated across a wide range of workloads and diverse precision levels at a 40% sparsity factor, achieving an application performance quality of 97.85% QoR.

1536-125X © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:26:07 UTC from IEEE Xplore. Restrictions apply.

TABLE I
SOTA MEMORY BIT CELL AND THEIR FEATURES AS IN-MEMORY COMPUTE ELEMENTS

	Tech. (nm)	Number of Transistors	Mult. Domain	Accuracy Support	Mult. Type
TCAS-I'25 [2]	65	14	Digital	-	AND
TCAS-I'24 [3]	55	8	Digital	High	MUX
JETCAS'22 [6]	65	8	Analog	Medium	AND
JSSC'21 [7]	65	NA	Digital	High	XNOR
TNANO'23 [8]	65	10	Digital	High	XNOR
JSSC'19 [14]	65	10	Analog	Medium	NA
Ours	65	8	Digital	High	AND

TABLE II
COMPARATIVE EVALUATION OF SOTA ADDER TREE WORKS

	Tech. (nm)	Adder Tree	Reconfigurability	Transistors/FA	Output Type
JSSC'21 [7]	65	Conventional	No	28T	Non-Inv.
JSSC'24 [9]	28	Hybrid†	Partial†	12T	Inverted
ISQED'25 [10]	65	Accurate RCAs	No	Interleaved (28T + 7T)	Non-Inv.
TCAS-I'25 [2]	65	PG-RCA	No	26T	Non-Inv.
TCAS-I'24 [3]	55	RCA + MUX	No	26T	Inverted
Ours	65	Optimised RCAs	Yes	14T FA + 11T HA	Non-Inv.

Note: Here, hybrid adder† tree approach refers to partial reconfigurability† between approximate compressors and accurate RCAs.

The remainder of the article is organised as follows. Section II discusses prior DCIM works and the motivation for this work. Section III outlines the fundamental elements of the proposed DCIM macro, including an M8T bit cell-based SRAM array and an area-efficient RCA with novel FA14 T and HA11 T. Section IV discusses the evaluation of reconfigurable DCIM macros, addressing the under-utilisation of memory banks and the impact on application performance. Finally, Section V concludes this work.

II. BACKGROUND AND MOTIVATION

A comprehensive evaluation of DCIM macros for NN workloads is needed to improve throughput and compute density. Tables I and II summarise the design features of SoTA bit cells and adder trees, along with their corresponding performance implications for DCIM macro design. AND-based MACs [2], [6] provide better reconfigurability and accuracy for a wide range of applications compared to binary XNOR-MACs [7], [8]. Table I primarily summarises the multiplication domain and operation types, number of transistors per bitcell, and accuracy implications for AI applications. Table II primarily compares prior works based on adder tree type, reconfigurability, output type, and the transistor count used to construct a single full adder within RCA, with implications for area and performance. Approximate adder-trees (DIMCA [9], [10], IMPLY [11], [12], DACA [13]) improve compute density and energy efficiency but overlook threshold-loss effects during multiplication, reducing noise margins. It also fails to provide runtime mechanisms to address macro under-utilisation via scalable precision and sparsity, leading to extra hardware and latency.

Prior work has seen improvements through algorithmic co-design [1], [3], architectural [7], [8], [9], [14], and circuit-level [6], [8], [15] optimisations to address challenges associated with computational energy, memory bandwidth, SNR, and throughput. 10 T CONV-SRAM bitcell [14] enhances the dynamic range by separating write and MAC operations, addressing write disturbance through charge-based accumulation and pulse-width control for multi-bit input, at the cost of extra transistors. The resolution of ADCs and DACs severely degrades storage density, energy efficiency, and computational accuracy. ACIM [6] mitigates threshold loss by limiting input voltages to between 0 V and 0.3 V, but DCIM's accuracy suffers under variable data. The counterpart DCIM macro eliminates the need for data-conversion hardware, supporting multi-precision reconfigurable computing [3], [7], at the minimal cost of additional shifters, adders, and accumulators. Prior works [9], [10], [16] explored area-optimised adder (reduction in transistor count) at the expense of accuracy loss. D6CIM [17] betters application performance with time-multiplexed hardware and fixed-point arithmetic precision, with additional inverters for signal and slew restoration. Sankhe et al. [10] explored interleaved adder trees with transistor-optimised FAs to minimise adder-tree area and limit voltage-drop propagation across stages. Yet, it fails to address voltage drops within the same RCA stage or threshold loss passed from the bitcell. Our proposed DCIM macro addresses most of these research gaps with our proposed approach.

III. PROPOSED DESIGN

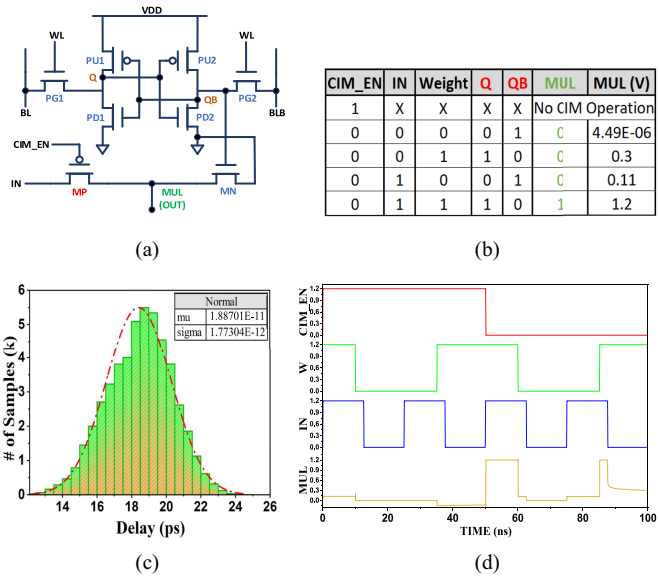
AI inference primarily focuses on matrix computations (MVM/MAC), which are typically performed within the DCIM macro. The fundamental elements of the proposed DCIM macro architecture include a novel M8T-based SRAM memory array, a controller, and an accumulator with an area-efficient adder tree. Fig. 4(a) depicts the detailed circuitry for the proposed 64 Kb macro. SRAM array is distributed into 64 banks, with 4 columns and 256 rows. The reconfigurable hierarchical adder tree is attached to the proposed CIM array and assists in summing up outputs from each row concurrently passed to the accumulator within a single clock cycle. The key elements include novel M8T, HA11 T, and FA14 T, which are discussed in forthcoming subsections.

A. M8T for Dual-Purpose CIM Array

The proposed M8T bit cell, as shown in Fig. 1(a), utilises two transistors (MP and MN) for independent multiplication operation with a conventional 6 T SRAM bitcell. With the CIM_EN signal, the pre-stored weights are multiplied by the input data whenever required, in addition to the traditional weight storage operation. The conventional 6T SRAM bitcell is used to store weight (W) data, and bitline (BL) and bitline bar (BLB) are pre-charged to VDD and enabled for the read operation. The input (IN) drives the source of the transistor MP; the multiplication output (MUL) is taken out from the common drain between transistors MP and MN during CIM mode. This avoids the need to turn on the wordline (WL) access transistor and eliminates the bit-flip issue caused by simultaneous activation of multiple rows. The proposed approach replaces 6 T CMOS AND logic with 2 T PTL AND. We used an MP transistor to ensure a full logical '1', with proper sizing as per [Eq.(1)] to ensure rail-to-rail voltage swing at MUL. This marks a key difference between the proposed dual-purpose SRAM array and the SoTA SRAM-enabled CIM array.

In CIM mode, the CIM_EN signal is low. If the input activation and the stored data are logic 1, the MP transistor will

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:26:07 UTC from IEEE Xplore. Restrictions apply.



CIM_EN	IN	Weight	Q	QB	MUL	MUL (V)
1	X	X	X	X	No CIM Operation	
0	0	0	0	1	0	4.49E-06
0	0	1	1	0	0	0.3
0	1	0	0	1	0	0.11
0	1	1	1	0	1	1.2

Fig. 1. (a) Schematic for Proposed M8T bit cell, (b) CIM operation table with MUL logical value (In green) and actual voltage values, (c) Monte-Carlo Simulations (Multiplication Delay) for proposed M8T Bitcell, (d) Simulation Waveform for Proposed M8T Bitcell.

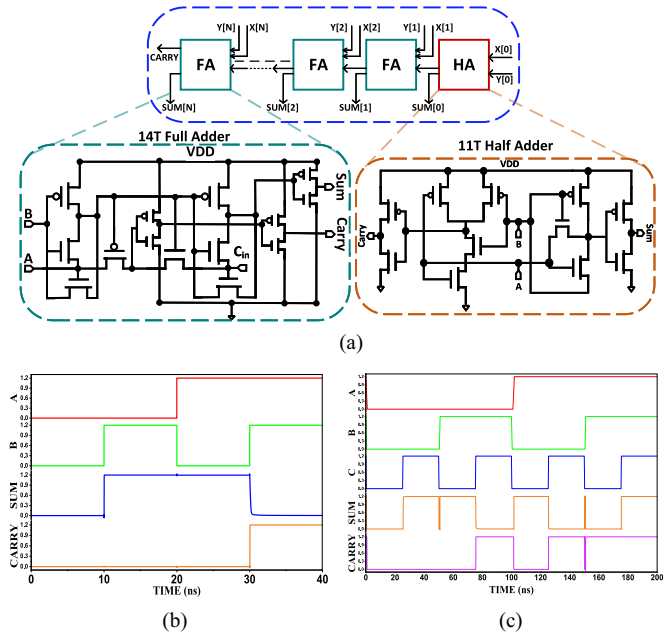


Fig. 2. (a) Proposed Ripple Carry adder with Schematic for 11 T half Adder and 14 T Full Adder, (b) Simulation Waveform for Proposed 11 T Half adder, (c) Simulation Waveform for Proposed 14 T full adder.

pass logic 1 to the MUL port without threshold loss, thereby performing the multiplication operation. Conversely, if the input activation and data stored are logic 0, the MN transistor pulls the MUL node to ground. If the input activation is logic 0 and the data stored is logic 1, then the MUL is discharged through the MP transistor. Similarly, when the input activation is logic 1, and the data stored is logic 0, the MUL node is pulled down to near logic 0 levels due to the adequately sized MN (1). This allows simultaneous weight updates when the input activation is 0, improving energy efficiency and throughput. Table I(b) showcases the detailed operation, including the logical and actual voltage levels, for multiplication between the Input and the Weight. The simulation waveform for the bit cell in all cases is shown in Fig. 1(d). The 8T SRAM bitcell occupies an area of 4.21 μm^2, $1.6\times$ larger than a conventional 6 T SRAM cell at a 65 nm CMOS technology node. The 100 K Monte-Carlo simulations for the multiplication delay are shown in Fig. 1(c), and the standard deviation for the delay demonstrated $4\times$ improvement over [17].

$$\left(\frac{W}{L}\right)_{\text{MN}} = 2 \times \left(\frac{W}{L}\right)_{\text{MP}} \tag{1}$$

The noise margin for the proposed M8T bit-cell is determined with the butterfly curves (Fig. 3) under hold and read operations at TT (27 °C) and SS (125 °C) corners, considering different modes: storage mode (CIM_EN off), active CIM, and comparison with a conventional 6T SRAM. The static noise margin (SNM) during active CIM operation remains comparable to that of the conventional 6 T cell, as QB is connected to the MN transistor gate, thereby preventing disturbance to the storage node. In storage mode or when CIM mode is disabled, the 2 T and transistors do not affect the storage node, and SNM remains within a few mV of the 6 T SRAM cell. In CIM mode, QB drives the MN transistor gate with a capacitive load, which weakens the noise margin slightly. In hold mode, the MN device gate appears to have high impedance from the QB node, but it still induces capacitance and parasitic coupling. This slows the inverter feedback loop at QB, weakening how strongly Q and QB nodes can reinforce each other against noise, thereby lowering the hold static noise margin (HSNM) by $\sim$ 50 mV and shrinking the lobe in the butterfly curve. While in read mode, the read noise margin is already degraded due to the access transistor connecting the internal nodes to bitlines, and further addition of a gate connection at QB (extra capacitance and parasitic coupling) has little influence compared to the strong read operation, which results in an read static noise margin (RSNM) drop of only $\sim$20 mV compared to the RSNM of a 6 T SRAM cell. The SNM analysis under asymmetric transistor strength conditions (FS/SF corners) exhibits trends consistent with those of TT/SS corners, indicating robust stability against pull-up/pull-down strength mismatch. The M8T bitcell demonstrates behaviour comparable to that of the conventional 6 T SRAM bitcell, confirming stable operation across process corners.

B. True Output HA11 T & FA14 T for Area-Efficient RCA

Partial product accumulation is performed using an adder tree in the DCIM array. However, it results in a significant loss in compute density compared to ACIM [6], [14], thereby making the accuracy-hardware tradeoff infeasible. Our work proposes an area-efficient RCA composed of HA11 T and FA14 T with transistor-level optimisation without affecting output functionality. The proposed schematics and simulation waveforms for the proposed HA11 T and FA14 T are shown in Fig. 2(a), (b), and (c), respectively. Conventional half-adder logic uses XOR for the sum and AND for the carry. In a CMOS-only

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:26:07 UTC from IEEE Xplore. Restrictions apply.

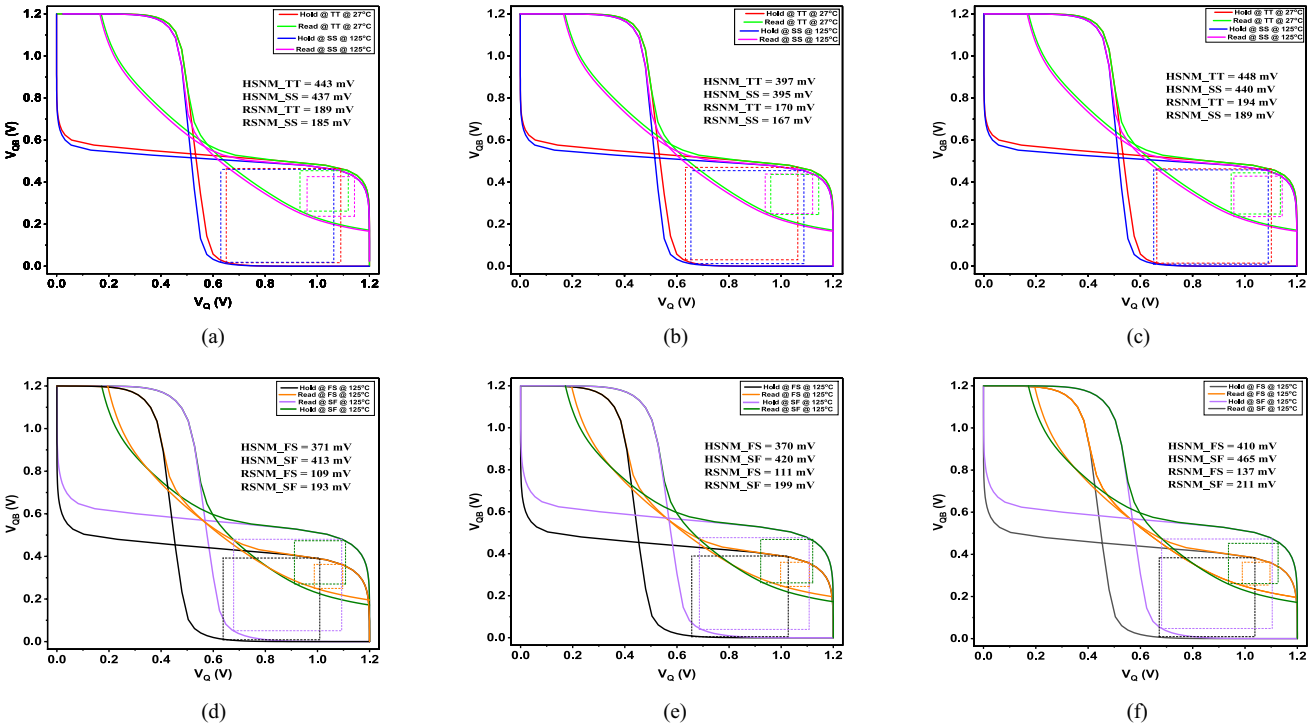


Fig. 3. Butterfly curves for the M8T bitcell for Hold and read operation (at TT(27 °C) and SS(125 ° C) corners) during (a) Storage mode (CIM_EN off), (b) Active CIM operation, (c) Conventional 6T SRAM, and (at FS(27 °C) and SF(125 °C) corners) during (d) Storage mode (CIM_EN off), (e) Active CIM operation, (f) Conventional 6 T SRAM.

design, the AND and XOR gates require 6 and 10 transistors, respectively. To reduce the transistor count, we adopted a 3 T PTL XNOR gate, followed by a 2 T CMOS inverter to realise the XOR-equivalent sum, ensuring full swing CMOS output. However, we retained CMOS logic in the carry path for minimal propagation delay. The ratioed PTL XNOR operation, based on transistor sizing and inverter threshold selection, has been confirmed with simulations with appropriate noise margin across the evaluated operating conditions.

Similarly, the conventional full adder is expressed as Sum $= A \oplus B \oplus C_{in}$, and $C_{out} = (A \oplus B) \cdot C_{in} + (A \cdot B)$. In the proposed design, the sum is obtained with 3 T PTL XNOR for A and B, followed by another PTL XNOR with C_{in} and output taken from with a 2 T CMOS inverter to restore logic levels. The first PTL XNOR introduces a drop of $(V_{\mathrm{DD}} - V_{\mathrm{Th}})$; however, as subsequent stages are connected to the NMOS source, the drop doesn't propagate further and is corrected by the CMOS inverter at the sum output of FA. For the carry, we utilised a logical equivalent with AND and XNOR operations between $A \odot B$ and A, followed by CMOS inversion and another XNOR with C_{in}. The carry path employs a 4 T buffer to ensure rail-to-rail swing, as it relies heavily on PTL logic. This buffer can be interleaved similarly to [10], reducing the transistor count by an additional 3.4%, which we identify as a potential direction for future work.

The use of a HA11 T at the least significant bit (LSB) is valid under the assumption of unsigned arithmetic accumulation, which was considered in our application scenario. For signed arithmetic, the LSB must support carry-in propagation arising from two's complement subtraction (i.e., Out $= X + (-Y) = X + \overline{Y} + 1$). Accordingly, the HA11 T at the LSB should be replaced with a FA14 T, thus enabling support for both two's complement arithmetic-based addition and subtraction. This modification ensures functional correctness for signed MAC operations without altering the overall adder-tree structure and incurs only marginal area and power overhead.

The functionality is validated through post-layout simulation, and it is observed that the adder tree consumes a tiny fraction of the total static power, accounting for only 6.13% of the dynamic power with the proposed approach. The proposed HA11 T and FA14 T occupy an area of 5.39 $\mu\mathrm{m}^2$, and 9.38 $\mu\mathrm{m}^2$ respectively, which is approximately $4.4\times$ and $2.25\times$ smaller, compared to conventional 28 T full adder at a 65 nm CMOS technology node. The proposed 4-bit and 8-bit RCA minimises area up to 36.45% and 38.2% compared to the conventional RCA adder. This will scale linearly, resulting in significant savings in the adder tree.

IV. Reconfigurable DCIM Macro Evaluation

The DCIM macro architecture focuses on critical components, such as the SRAM memory array, an adder tree, a controller and peripheral circuitry, and an accumulator. Our dual-purpose DCIM array and area-efficient RCA improve throughput by increasing compute density and reducing power and delay, meeting the needs of improved edge devices. The overall architecture of the proposed 64 Kb DCIM macro is illustrated in Fig. 4(a). The dual-purpose DCIM array is programmed in a row-wise weight-stationary approach, assisted by a CIM_EN driver and selection multiplexers. The inputs are fed bit-serial from LSB to MSB, generating partial products on every clock

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:26:07 UTC from IEEE Xplore. Restrictions apply.

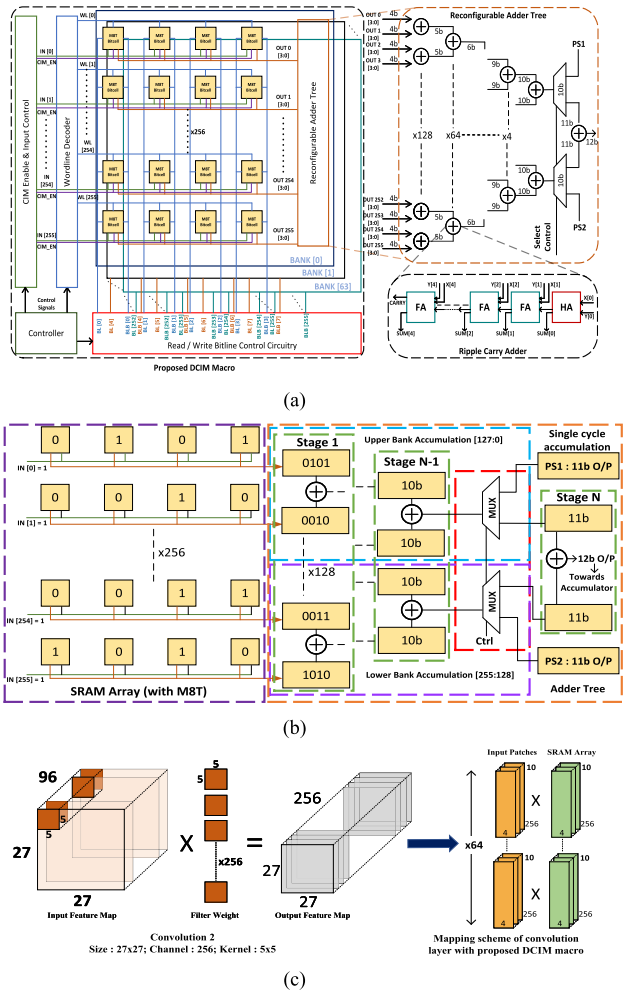


Fig. 4. Illustrations of the proposed DCIM macro and its usage: (a) Proposed DCIM Macro, (b) Illustration of 1A4W MAC Computation with the Proposed DCIM Macro, showcasing single and dual-bank accumulation, (c) Convolution Layer (Conv. 2) Mapping for the Proposed DCIM Macro.

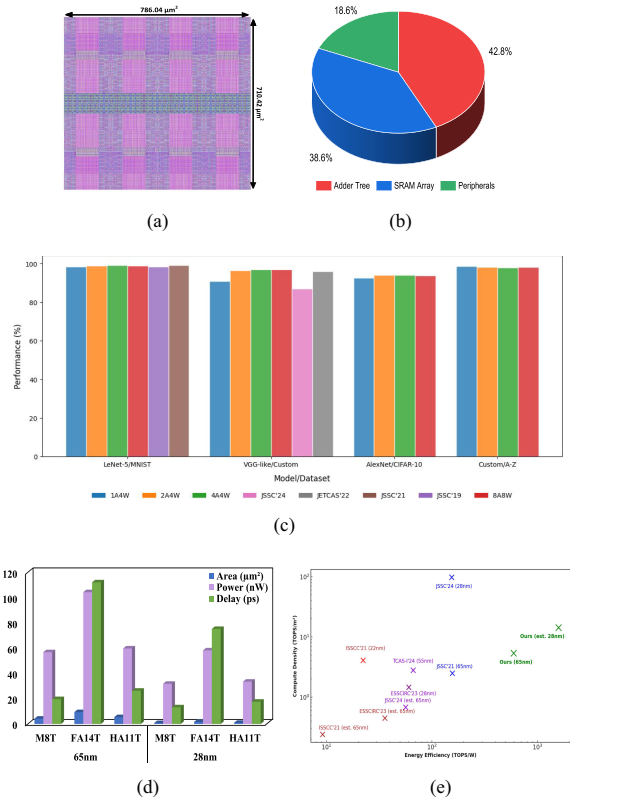


Fig. 5. (a) Layout of proposed DCIM macro, (b) Area Utilisation Breakdown of DCIM Macro, (c) Comparison for Inference accuracy with simulated Macro[1] compared with SoTA works [6], [7], [9], [14], (d) Impact of scaling to 28 nm on performance parameters (area, power, and delay) for M8T, FA-14 T, and HA-11 T, (e) Performance analysis for proposed DCIM macro with SoTA works [3], [7], [9], [15], [17].

cycle. The weight precision is limited to 4-bit and can be scaled in multiples of 4 using multiple banks, while the input bit-width is adjustable as required, requiring N computation cycles for N-bits. The partial products for each kernel are computed within a single SRAM bank and passed to an adder tree for a 12-bit sum in single-bank accumulation. The MAC result is calculated from the partial sums using an accumulator circuit over N+1 clock cycles. A detailed illustration of MAC computations is shown in Fig. 4(b), where multiplication is performed with an M8T-based SRAM array, followed by single-cycle accumulation using the adder tree. The detailed illustration of the proposed macro for computing 1A4W in both single- and dual-accumulation approaches is shown in Fig. 4(c).

A. Reconfigurable Adder Tree for Incentivised Performance

AI model evaluations showcased that the DCIM macro is underutilised. Thus, we evaluated the software for algorithms that support resource-friendly, low-latency, and flexible precision computations, providing well-balanced optimisations across multiple metrics and delivering a performance-enhanced, lightweight DCIM macro. For the AI workloads evaluated, as shown in Fig. 4(c), we observed that the 40 % sparsity factor does not affect application performance, thereby enabling us to accelerate DNN workloads with enhanced energy efficiency and compute density. However, this results in fewer consecutive partial products than the available rows in the DCIM array, and a single accumulation per memory bank. Thus, to address these issues, we decided to divide the adder tree into two parts of order

TABLE III
ALEX NET MAPPING

Layer	Kernel (Kh x Kw x Cin)	fmaps (Ho x Wo x Co)	MACs ($x10^6$)	Param.s ($x10^6$)	Banks	Op. Cycles ($x10^3$)
Input	-	224 x 224 x 3	NA	-	NA	-
Conv. 1	11 x 11 x 96 (Stride = 4)	55 x 55 x 96	105.42	0.035	96	9
MP1	3 x 3 (Stride = 2)	27 x 27 x 96	-	-	NA	-
Conv. 2	5 x 5 x 256 (Stride = 2)	27 x 27 x 256	223.95	0.31	4374	27
MP2	3 x 3 (Stride = 2)	13 x 13 x 256	-	-	NA	-
Conv. 3	3 x 3 x 384	13 x 13 x 384	149.52	0.89	1140	1.5
Conv. 4	3 x 3 x 384	13 x 13 x 384	112.14	0.67	2432	3
Conv. 5	3 x 3 x 256	13 x 13 x 256	74.76	0.44	2432	3
MP3	3 x 3 (Stride = 2)	6 x 6 x 256	-	-	NA	-
Flatten	NA	9216 x 1	-	-	NA	-
FC 1	4096	4096	37.75	37.75	4096	590
FC 2	4096	1000	16.78	16.8	2624	262
FC 3	1000	1	4.1	16	4	66

Note: For 1A4W, it takes 1 clock cycle per memory bank, as demonstrated in Fig. 4 (b). For 1A8W, two memory banks are required. For $2A4W$, it takes 2 clock cycles, while $4A4W$ requires 4 clock cycles (from LSB to MSB).

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:26:07 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
PERFORMANCE COMPARISON WITH SOTA ACIM AND DCIM MACROS

Parameter	ESSCIRC'23 [17]	JSSC'19 [14]	JETCAS'22 [6]	TNANO'23 [8]	JSSC'21 [7]	ISSCC'21 [15]	JSSC'24 [9]	TCAS-I'24 [3]	Ours
Tech. (nm)	28	65	65	65	65	22	28	55	65
MAC Operation	Digital	AMS	Analog	Digital	Digital	Digital	Digital	Digital	Digital
Cell Type	6T+0.5T	10T	AND8T	10T	NA	6T	8T	8T	M8T
Supply Voltage (V)	0.6-1.1	0.8-1.0	1	1.2	0.6-0.8	0.72	0.45-1.1	1.2	1.2
Array Size (Kb)	16	16	16	16	16	64	16	64	64
Op. Frequency (MHz)	200	5	100	25	138	100	250	200	250
Bitcell Area (μm^2)	0.39	NA	2.78	4.5	10.53	0.37	NA	4.28	4.21
Macro Area (mm^2)	0.59, 3.43*	NA	NA	NA	0.23	0.22, 1.87*	0.49, 2.84*	2.8	0.87
Input Precision (bit)	1-8	6	4	4	1-16	1-8	1-4	4/8/12/16	1-16
Weight Precision (bit)	8	1	4	1-4	1-16	4/8/12/16	1	4/8/12/16	4/8/12/16
Model	NA	LeNet-5	VGG-8	CNN-Type	LeNet-5	NA	VGG-8	ResNet-10	AlexNet
Accuracy	NA	98.3	96.05	98.67	99.2	NA	86.96	93.7	94.2
Throughput (TOPS)	2.40, 1.62†	0.64	0.41	0.82	0.56	0.83, 0.52†	4.8, 2.97†	1.28	4.67, 1.93‡

Note: Equivalent Macro area* is normalized quadratically and Throughput† at CMOS 65nm, with Tech Scaling [8], [9], [14], [15], [17]; Scaled throughput‡ for 16Kb of proposed DCIM macro, for a fair comparison with 16Kb Macros, CMOS 65nm [6], [8], [14].

2, with multiplexers at the N-1 stage and a dual-accumulation approach (two sub-banks). This allows the same DCIM array to be utilised for two MAC computations in subsequent computation cycles, with a slight delay, and for partial products to be accumulated from PS1 and PS2, respectively (Fig. 4(b)). This is more beneficial when combined with 40% pruning [10]; the effective computational workload drops significantly, and the utilisation improves further. This results in a reduction in operation cycles by $3.2\times$ and in memory bank usage by 30%, thus providing significant acceleration without compromising accuracy and enhanced throughput over prior works.

B. Evaluation for DNN Performance

The proposed DCIM macro targets edge-node real-time multi-character recognition with custom, additional 8-layered multi-class (45) support vector classifiers (SVC). The detailed neural network mapping for the Alex-Net architecture, including DCIM memory bank utilisation and operation cycles, is reported in Table III. A parameterised software evaluation model with $QKeras$ library using $Python$ 3.0 on the $Google\ Colab$ platform and extracted weights in CSV format for manual mapping to DCIM macro. The detailed precision with macro utilisation tradeoff is shown in Fig. 5(c), which involves evaluation for LeNet-5/MNIST, VGG-like/Custom dataset, AlexNet/CIFAR-10 and Custom-8/A-Z Handwritten dataset at 1A4W, 2A4W, 4A4W and 8A8W and is compared with FP32-tensor and SoTA works [6], [7], [9], [14]. The evaluation showcases the performance with 97.85% QoR compared to FP32 as reported in [1] and outperforms SoTA works [6], [7], [9], [14] with a significant improvement up to $0.5-4$ % in application accuracy.

The hardware performance emulation model is defined by the following design parameters: DCIM array dimensions, activation and weight precision, the number of memory banks required for layer execution, and reconfigurable hardware multiplexing. Table III illustrates the memory mapping for automatically loading Alex-Net kernels layer by layer in the proposed macro with macro mapping for Conv. 2 shown in Fig. 4(c). The input feature-maps to the layer are $27 \times 27 \times 96$, which will be convoluted with 256 kernels of 5×5, thereby producing 27x27x256 output feature maps. Each 4-bit kernel requires 10 banks to be computed, and 64 simultaneous partial products are generated every clock cycle. The macro is required to reuse the bank 4374 times for over 27 k Op. cycles. The proposed DCIM macro is utilised by reprogramming dense weights layer by layer, featuring an off-chip bit-serialised ReLU activation function and control engine circuitry, as follows [6]. The result combination accumulator, sub-sampling layer, softmax non-linear layer, and normalisation are considered outside the DCIM macro. This work evaluated the sparsity-aware layer-reuse philosophy, maintaining kernel-wise modularity and macro scalability across various network configurations, thereby enhancing performance for resource-intensive AI applications.

C. Performance Metrics Analysis

The implications of macro size, precision, and throughput for the proposed DCIM macro compared to SoTA works are discussed below to quantify the impact of this work. The proposed 64 kb macro achieves $5.4\times$ and $3.6\times$ improvement in throughput compared to prior works [3] and [15], respectively. Our DCIM macro provides additional input precision support from 1 to 16 bits and weight precision to 4, 8, 12, or 16 bits, whereas the prior works limit input precision to 1 to 8 bits in [15] and 4, 8, 12, or 16 bits in [3] for similar weight precision support. The scaled proposed 16 Kb macro shows $4.6\times$, $2.4\times$ and $1.2\times$ throughput improvements compared to prior works [6], [8], and [17] respectively. However, the weight/input-precision support in these works is limited to 4-bit/4-bit [8], 4-bit/(1-4)-bit [6], and 8-bit/(1-8)-bit [17], respectively. In contrast, our macro supports $(1-16)$-bit input and $4/8/12/16$-bit weight precision. VGG-like/Custom refers to a custom 8-layer network trained on a custom dataset (formed from selective images from MNIST, SVHN, and Car license plate detection). The increased throughput is primarily contributed to by higher operation frequency and reduced AI workload due to sparsity exploitation.

The layout and area breakdown of the proposed DCIM macro at the CMOS 65 nm technology node are illustrated in Fig. 5(a) and (b), respectively. The primary analysis focused on energy efficiency (TOPS/W) and compute density (TOPS/mm^2), which assists in assessing the computational power of the device. The key factor driving the enhancement in compute density is the incorporation of an area-efficient reconfigurable adder tree for efficient macro utilisation with a 40% sparsity factor. In contrast, the lower energy consumption is driven by design optimisation, such as reduced static power consumption and separation of CIM

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:26:07 UTC from IEEE Xplore. Restrictions apply.

operation. The proposed DCIM macro achieves $16.7\times$, $9.2\times$ and $3.8\times$ better energy efficiency compared to [3], [17], and [7] respectively. The enhancements in compute density are by $8.6\times$, $4.1\times$ and $1.9\times$ compared to [15], [17], and [7] respectively.

The proposed M8T bit-cell, FA-14 T, and HA-11 T functions as intended when implemented with CMOS 28 nm and comparison for the performance metrics (area, power, and delay) is reported in Fig. 5(d), which translates the macro-level performance improvements up to $1.49\times$, $2.67\times$, and $2.2\times$ in throughput, compute density, and energy efficiency, respectively, compared to 65nm. Technology scaling and macro size are adapted as detailed in Table IV and Fig. 5(e) for fair comparison. The scaled proposed macro (28nm CMOS) estimates $14.7\times$, $9.6\times$ and $6.3\times$ higher energy efficiency compared to [15], [17], and [9] respectively and higher compute density by $8.6\times$, and $3.2\times$ compared to [17], and [15] respectively. Increased compute density signifies a compact, cost-effective design that enables more computations within a smaller silicon area. Enhanced energy efficiency prolongs operational life in resource-constrained platforms while reducing operational costs associated with cooling and power consumption at cloud nodes.

V. Conclusion

This work introduces RAPID-CIM, a reconfigurable, energy-efficient SRAM-based DCIM macro to address SoTA edge-AI acceleration challenges. The proposed macro achieves up to $16.7\times$ energy efficiency, $8.6\times$ compute density, and a $3.2\times$ reduction in operation cycles compared to prior works, utilising a novel M8T bitcell and a scalable, bit-reconfigurable adder tree. The proposed macro is evaluated at CMOS 65 nm and also demonstrated to be scalable to CMOS 28 nm. The detailed evaluations of AlexNet, LeNet-5, and custom multi-class classification models demonstrate improvements in accuracy over SoTA methods with minimal resource overhead. Thus, the proposed solution is better suited for compact battery-powered edge devices.

References

[1] M. Lokhande, G. Raut, and S. K. Vishvakarma, "Flex-PE: Flexible and SIMD multiprecision processing element for AI workloads," *IEEE Trans. Very Large Scale Integration Syst.*, vol. 33, no. 6, pp. 1610–1623, Jun. 2025.

[2] V. Sharma, X. Zhang, N. S. Dhakad, and T. T. -H. Kim, "FlexDCIM: A 400 MHz 249.1 TOPS/W 64 kb flexible digital compute-in-memory SRAM macro for CNN acceleration," *IEEE Trans. Circuits Syst. I: Reg. Papers*, vol. 72, no. 10, pp. 5696–5707, Oct. 2025.

[3] W. Yi et al., "RDCIM: RISC-V supported full-digital computing-in-memory processor with high energy efficiency and low area overhead," *IEEE Trans. Circuits Syst. I: Reg. Papers*, vol. 71, no. 4, pp. 1719–1732, Apr. 2024.

[4] S. Kim and H. J. Yoo, "An overview of computing-in-memory circuits with DRAM and NVMs," *IEEE Trans. Circuits Syst. II: Express Briefs*, vol. 71, no. 3, pp. 1626–1631, Mar. 2024.

[5] O. Krestinskaya, L. Zhang, and K. N. Salama, "Towards efficient in-memory computing hardware for quantized neural networks: State-of-the-art, open challenges and perspectives," *IEEE Trans. Nanotechnol.*, vol. 22, pp. 377–386, 2023.

[6] V. Sharma, J.-E. Kim, H. Kim, L. Lu, and T. T.-H. Kim, "A reconfigurable 16 Kb AND8T SRAM macro with improved linearity for multi-bit compute-in memory of artificial intelligence edge devices," *IEEE Trans. Emerg. Sel. Topics Circuits Syst.*, vol. 12, no. 2, pp. 522–535, Jun. 2022.

[7] H. Kim, T. Yoo, T. T.-H. Kim, and B. Kim, "Colonnade: A reconfigurable SRAM-based digital bit-serial compute-in-memory macro for processing neural networks," *IEEE J. Solid-State Circuits*, vol. 56, no. 7, pp. 2221–2233, Jul. 2021.

[8] P. K. Saragada, S. Manna, A. Singh, and B. P. Das, "A configurable 10 T SRAM-based IMC accelerator with scaled-voltage-based pulse count modulation for MAC and high-throughput XAC," *IEEE Trans. Nanotechnol.*, vol. 22, pp. 222–227, 2023.

[9] C.-T. Lin et al., "DIMCA: An area-efficient digital in-memory computing macro featuring approximate arithmetic hardware in 28nm," *IEEE J. Solid-State Circuits*, vol. 59, no. 3, pp. 960–971, Mar. 2024.

[10] A. Sankhe, M. Lokhande, R. Sharma, and S. K. Vishvakarma, "Area-optimized 2D interleaved adder tree design for sparse DCIM edge processing," in *Proc. 26th Int. Symp. Qual. Electron. Des.*, 2025, pp. 1–6.

[11] N. Kaushik and S. Bodapati, "IMPLY-Based high-speed conditional carry and carry select adders for in-memory computing," *IEEE Trans. Nanotechnol.*, vol. 22, pp. 280–290, 2023.

[12] J. Huang, T. N. Kumar, H. A. F. Almurib, and F. Lombardi, "On the commutative operation of approximate CMOS ripple carry adders (RCAs)," *IEEE Trans. Nanotechnol.*, vol. 23, pp. 265–273, 2024.

[13] X. Fan, T. Zhang, H. Li, H. Liu, S. Lu, and J. Han, "DACA: Dynamic accuracy-configurable adders for energy-efficient multi-precision computing," *IEEE Trans. Nanotechnol.*, vol. 22, pp. 400–408, 2023.

[14] A. Biswas and A. P. Chandrakasan, "CONV-SRAM: An energy-efficient SRAM with in-memory dot-product computation for low-power convolutional neural networks," *IEEE J. Solid-State Circuits*, vol. 54, no. 1, pp. 217–230, Jan. 2019.

[15] Y.-D. Chih et al., "An 89TOPS/W and 16.3TOPS/mm2 all-digital SRAM-Based full-precision compute-in memory macro in 22 nm for machine-learning edge applications," in *Proc. IEEE Int. Solid-State Circuits Conf.*, 2021, pp. 252–254.

[16] V. Gupta, D. Mohapatra, A. Raghunathan, and K. Roy, "Low-power digital signal processing using approximate adders," *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, vol. 32, no. 1, pp. 124–137, Jan. 2013.

[17] J. Oh, C.-T. Lin, and M. Seok, "D6CIM: 60.4-TOPS/W, 1.46-TOPS/mm^2, 1005-Kb/mm^2 digital 6T-SRAM-Based compute-in-memory macro supporting 1-to-8b fixed-point arithmetic in 28-nm CMOS," in *Proc. IEEE Eur. Solid-State Circuits Conf.*, 2023, pp. 413–416.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:26:07 UTC from IEEE Xplore. Restrictions apply.